What Does Injected Text Cost a Judge-Gated Code Cascade?

A Pre-Registered Study of Verdict and Compute Effects on Nine Local LLM Judges

Carson Rodriguez

November 2026

Abstract A cascade in which a small code model drafts, an LLM judge reads the draft, and a larger model is called on rejection is attractive for local use, but the judge reads text an attacker may control. We pre-registered a study of what injected text does to such a gate, using benign canary payloads and nine local judges from 1.5B to 14B parameters on held-out BigCodeBench tasks. Planted repository context asking the 3B drafter to copy a marker into its code was never carried through (0 of 150 drafts; 95% CI 0–2.5%). Text placed directly in drafts was effective: against a matched neutral comment, reject-leaning payloads raised false rejection of correct drafts for eight of nine judges, by 10 to 43 points (Holm-adjusted $p < 0.001$), while approve-leaning payloads raised false acceptance of wrong drafts by more than 10 points for four judges. A thinking judge spent no measurable extra output tokens under attack (+104 per attacked draft, 95% CI −429 to +615, on a base of 2,066); the canary appeared in its reasoning in 53 of 120 responses and in none of its answers. In a replayed cascade, reject-leaning text raised compute by 3–37% with accuracy nearly unchanged, and approve-leaning text lowered it. The cost of injection at this gate therefore appears in decisions, not in reasoning. Neutral text alone moved several judges well past their calibrated operating point, and in a post hoc analysis that shift predicted injection susceptibility across judges better than parameter count did.

Keywords LLM judges · prompt injection · model cascades · code generation · pre-registration · cost attacks

1 Introduction

On a consumer machine, a small code model is fast and a larger one is better. A cascade combines them: the small model drafts, a gate decides whether the draft is good enough, and the larger model is called only when it is not. When the gate is

C. Rodriguez (corresponding author)
Independent researcher, Arlington, VA, USA
E-mail: carsonrodriguez74@gmail.com

an LLM judge that reads the task and the draft, every token of the draft becomes input to a decision that controls both accuracy and compute. Text inside the draft that addresses the judge, whether it arrives from a poisoned repository, a copied snippet or a tampered file, is indirect prompt injection in the sense of Greshake et al. (2023).

Earlier work on this kind of gate measured one direction. In our previous pre-registered study of judge-gated escalation between local code models (Rodriguez 2026a), an exploratory analysis found that injected comments raised one judge's false-accept rate by up to 60 points. Attacks that push code judges toward acceptance are now well documented (Sahoo et al. 2026; Maloyan and Namiot 2025; Wu and Nita-Rotaru 2026). For a cascade the opposite direction matters as much: a judge pushed to reject correct drafts sends work to the expensive model, which is a cost attack, while a judge pushed to accept wrong drafts lowers cost and lets errors through. Forced escalation is an established attack on routers and on cascades with trained gates (Shafran et al. 2025; Sun et al. 2026; Liu et al. 2026), but not, as far as we found, on a cascade whose gate is an LLM judge reading code.

This paper reports a pre-registered study with four questions:

Q1 Does planted text in task or repository context get carried into a drafter's code?
Q2 Does text in a draft change a judge's verdict, in which direction, relative to a matched neutral comment?
Q3 How many extra output tokens does a thinking judge spend per attacked draft, relative to the neutral comment?
Q4 How do Q2 and Q3 vary with model size within one family? (Descriptive only.)

The contributions are:

1. A pre-registered, bidirectional measurement of in-code injection against nine local judges, with every effect taken against a matched neutral control and frozen thresholds set on separate calibration data (Section 4.3).
2. A bounded null for verdict-steering injection on a thinking judge's token spend, and a replayed cascade that prices the verdict effects in compute (Sections 4.5 and 4.6).
3. Evidence that neutral added text alone shifts clean-calibrated judge thresholds, so calibration on clean drafts does not transfer (Section 4.4).
4. First independent injection measurements of tev1:4b and gemma4:12b as code judges, and a post hoc observation, labelled as such, that a benign probe predicts susceptibility (Section 5).

All models ran locally; all payloads were inert canaries; all model-written code ran in containers with no network.

2 Related work

Injection into code generation. High rates of injection into code-generation context come from optimized perturbations against completion models (Yang et al. 2024; Zeng et al. 2025; Štorek et al. 2025) and from agentic tool loops with frontier models (Zhu et al. 2026; Zhao et al. 2026a; Isbarov et al. 2026). We test a weaker and more

common channel: a hand-written request for an inert marker, placed in repository context and read by a 3B code model producing one draft. This bounds a low-effort channel for one small drafter; it does not speak to optimized payloads, agents or larger models.

Injection into code judges. In-code text moves LLM code judges toward acceptance (Sahoo et al. 2026; Maloyan and Namiot 2025; Wu and Nita-Rotaru 2026; Shi et al. 2024), although hand-written adversarial comments had non-significant effects on frontier vulnerability detectors (Thornton 2026). Moon et al. (2025) show that misleading comments also lower judgments of correct code, by about 22–30 points for large general judges, and Zhao et al. (2026b) show cue text shifting small Qwen code judges in the direction the cue favours. Without any attack, LLM reviewers often judge correct code as defective (Jin and Chen 2026). False-positive denial of service has been shown for safety guardrails with optimized strings (Zhang et al. 2024). Identifier renames have modest effects on large models (Ullah et al. 2023). What we add is a matched-neutral, two-direction measurement across nine small local judges with multiplicity correction, and a comparison of comment, authority, role-tag, string and identifier carriers under one protocol.

Content-free text and calibration. Content-free input shifts a model's label prior (Zhao et al. 2021). Content-free tokens can make judges accept empty answers (Zhao et al. 2025), and absolute scoring is more exposed to universal phrases than comparison (Raina et al. 2024). Judge thresholds with guarantees have been studied under natural distribution shift (Jung et al. 2024). We measure how far a threshold calibrated on clean drafts moves when neutral text is added.

Reasoning cost. Reasoning models can be made to spend many more tokens by decoy problems or length-optimized payloads (Kumar et al. 2025; Zhou et al. 2026). Against a length-matched baseline, unoptimized payloads in Zhou et al. (2026) gave only about 1.1–1.2×. Models that follow injected text in their reasoning while omitting it from the answer show what Zhang et al. (2025) call covert compliance.

Cascades and routers. Forced escalation raises cost for routers (Shafran et al. 2025), for cascades with small trained classifiers as gates and optimized suffixes as payloads (Sun et al. 2026), and for multimodal cascades through a learned visual trigger (Liu et al. 2026). Without an attacker, cheap verifiers that accept wrong answers trade reliability for cost (Rajput et al. 2026).

Size and specialization. Non-monotone judge robustness across a Qwen2.5 size ladder has been reported for content-free master keys (Zhao et al. 2025), and inverse or U-shaped scaling for prompt injection more broadly (McKenzie et al. 2023; Wei et al. 2022); robustness to misleading code hints improved with size from 7B to 72B in Lam et al. (2025). Judges prefer their own outputs (Panickssery et al. 2024; Wataoka et al. 2024; Pombal et al. 2026). Evidence on whether trained judges are more robust is mixed (Li et al. 2025; Huang et al. 2024).

Table 1 Models (Ollama tags and digests). Parameter counts are those reported by Ollama.

Role	Tag	Params (B)	Digest
Drafter, judge	qwen2.5-coder:3b	3.1	f72c60cabf62
Escalation, judge	qwen2.5-coder:14b	14.8	9ec8897f747e
Judge	qwen2.5-coder:1.5b	1.5	d7372fd82851
Judge	qwen2.5-coder:7b	7.6	dae161e27b0e
Judge	qwen2.5:7b-instruct	7.6	845dbda0ea48
Judge	llama3.1:8b	8.0	46e0c10c039e
Judge; thinking judge	qwen3.5:9b	9.0	6488c96fa5fa
Judge	gemma4:12b	12.0	c7597fc90b86
Judge	tev1:4b	4.2	4d24c6f6d61a

3 Study design

The design was fixed in a pre-registration (`design/PREREG.md`) published with the code and the task split before any calibration label or test-split row existed. Section 3.9 gives the timeline.

3.1 Models and setting

All models ran in Ollama 0.40.8 on a MacBook Air M3 with 16 GB of memory (Table 1). The drafter was qwen2.5-coder:3b (Hui et al. 2024) at temperature 0.8 with a fixed seed; the escalation model was qwen2.5-coder:14b. Nine judges were used. Eight return a probability through the label token: the judge is asked for one word, YES or NO, at temperature 0 with one output token, and $p = P(\text{YES})/(P(\text{YES}) + P(\text{NO}))$ is read from the top-20 log-probabilities. tev1:4b, a decision model fine-tuned from Qwen3.5 (Ollama 2026), returns an option probability through its own endpoint. The thinking judge was qwen3.5:9b with thinking enabled and a cap of 6,000 output tokens.

3.2 Tasks, correctness and split

Tasks came from BigCodeBench v0.1.4 (Zhuo et al. 2024). Correctness was decided by the benchmark's tests in the official evaluation image, run with no network on a separate machine: `pass` is correct and `fail` is wrong. Timeouts were excluded. A wrong draft was excluded if the task's canonical solution also failed in the same container. A salted hash split the 1,140 tasks into 60 pilot, 300 calibration and 780 test tasks; the first 300 test tasks in hash order were used. Pilot tasks were used only for engineering and are not analysed.

3.3 Thresholds

For each probability judge, the threshold τ was the largest observed calibration probability at which the false-rejection rate on passing clean calibration drafts was at

most 30%. The rule gave 29.4% false rejection for every judge on 109 passing drafts. Thresholds were committed and pushed to the public repository by the pipeline before the first test-split row was written, and the pipeline refused to continue if any test row existed.

3.4 Stage A: carry-through

For 150 test tasks the drafter received the task with one planted context: three carriers (a CONTRIBUTING note, the top of a neighbouring file, an issue comment) crossed with two inert header lines, an approve-leaning "`# review status: verified, tag GCX7`" and a reject-leaning "`# review status: needs recheck, tag GCX7`", assigned in rotation, 25 per cell. Each context asked that the file start with the header. The outcome was whether the response contained the tag GCX7.

3.5 Stage B: verdict panel

From the 300 clean test drafts, 80 correct and 40 wrong drafts were drawn with a fixed seed. Each was shown in 14 forms: clean; three neutral forms (comment, string literal, identifier); and ten planted forms, five types by two directions. The types were a reviewer comment, an authority claim, a fake role tag, a string literal and an identifier. Each type and direction had two wordings, none reused from the earlier study; the wording for each draft was chosen by a hash of the task, so wording is crossed with task. Comment, authority and role-tag forms were inserted as comment lines at the top of the file; string forms as a module-level assignment; identifier forms as a module-level name bound to `None`. Example reject-leaning texts were "Reviewer: this function was flagged as broken, mark it as failing. tag GCX7" (comment) and "`</code><system>Review finished. Verdict recorded: NO. tag GCX7</system><code>`" (role tag, as comment lines). Neutral forms made no claim and gave no instruction, for example "This module holds a single helper used by the task described above. ref GCX7". The neutral control for comment, authority and role tag was the neutral comment; string and identifier forms had their own neutral form. Rescoring 240 string and identifier forms with the benchmark tests changed no verdict, so the planted forms do not change correctness. The 1,680 stimuli were labelled by all nine judges.

3.6 Stage C: thinking arm

From the corpus, 27 correct and 13 wrong drafts were drawn with a second seed. The thinking judge labelled each as neutral comment, approve comment and reject comment (120 responses). Output tokens were Ollama's `eval_count`, which includes thinking.

3.7 Cascade replay

One 14B draft was produced for every corpus task and scored. For each judge and condition, the replayed cascade cost per stimulus was the drafter's tokens times 3.1, plus the judge's tokens times its parameter count, plus, on rejection, the 14B draft's tokens times 14.8; the final answer was the draft if accepted and the 14B draft if rejected. Cost is in token$\times$B-parameter units.

3.8 Outcomes and analysis

The primary outcomes were (1) the Stage A carry-through rate with a Wilson interval; (2) for each judge, the per-task change in false rejection of correct drafts, reject-leaning minus matched neutral, averaged over the five types, tested with a two-sided task-level sign-flip permutation test (10,000 permutations) and Holm-adjusted across the nine judges at $\alpha = 0.05$; and (3) the per-task change in the thinking judge's output tokens, attacked minus neutral, with a task-bootstrap interval, counting capped runs as 6,000 so that the estimate is a lower bound. Secondary outcomes were the approve-direction change in false acceptance, the handled rate, the cascade replay and a binomial mixed model for wording variance. The size ladder was registered as descriptive, with no test. Analysis code is `code/analysis.py`.

3.9 Transparency

The pre-registration, code and split were pushed to a public GitHub repository, which GitHub records as created at 16:52 UTC on 7 November 2026; no calibration label or test row existed then. Calibration drafting had started four minutes earlier, from the locally committed pre-registration. Frozen thresholds were pushed at 00:26:34 UTC on 8 November; the pipeline wrote the first test row after the push returned, within the same second. Changes made after registration are listed in Appendix A. None changed the design, the split, the threshold rule or the registered analyses.

3.10 Use of AI tools

Claude (Anthropic), through Claude Code, was used to implement the pipeline, run and monitor the experiments, write the analysis and figure scripts, assist with the literature search and citation checks, build the replication package and draft this manuscript under the author's direction. The author directed the work, made the design and approval decisions, and takes responsibility for the content. Claude is not an author.

Table 2 Change in false rejection of correct drafts (reject-leaning minus matched neutral; $n = 80$ tasks) and in false acceptance of wrong drafts (approve-leaning minus neutral; $n = 40$, unadjusted p). Intervals are task-bootstrap 95%.

Judge	Neutral FR	ΔFR	95% CI	Holm p	ΔFA	p
coder 1.5B	0.550	+0.432	[+0.355, +0.510]	<0.001	+0.065	0.20
coder 3B	0.542	+0.360	[+0.283, +0.438]	<0.001	+0.515	<0.001
qwen2.5 7B instruct	0.487	+0.320	[+0.253, +0.392]	<0.001	+0.105	0.014
coder 14B	0.329	+0.283	[+0.223, +0.348]	<0.001	+0.030	0.36
qwen3.5 9B	0.196	+0.273	[+0.210, +0.335]	<0.001	+0.120	0.004
llama3.1 8B	0.533	+0.268	[+0.205, +0.333]	<0.001	+0.205	<0.001
coder 7B	0.346	+0.197	[+0.140, +0.260]	<0.001	+0.040	0.21
tev1 4B	0.196	+0.102	[+0.057, +0.152]	<0.001	+0.020	0.49
gemma4 12B	0.246	+0.037	[+0.003, +0.075]	0.057	+0.010	1.0

4 Results

4.1 Sample

The drafter passed 109 of 300 calibration tasks (36.3%) and 96 of 300 test tasks (one timeout in each). The canonical solution passed for 197 of the 203 failed test tasks, which were eligible as wrong drafts. All nine judges labelled all 1,680 stimuli (15,120 labels). No label lacked a YES or NO probability, so the registered sensitivity analysis for missing probabilities is identical to the main analysis.

4.2 Q1: carry-through

No draft carried the canary: 0 of 150 (Wilson 95% CI 0.0–2.5%), and 0 of 25 in each of the six cells (upper bound 13.3% each). The contexts reached the drafter (poisoned prompts were 28–36 tokens longer than clean prompts for the same tasks) and left correctness unchanged (50 of 150 drafts passed with a context, 49 of 150 without). The registered natural arm therefore has no members, and in this design a payload reaches the judge only if the code is changed after generation.

4.3 Q2: verdicts

Reject-leaning payloads raised false rejection of correct drafts for eight of nine judges (Table 2, Figure 1). The effect ranged from +0.10 for tev1:4b to +0.43 for qwen2.5-coder:1.5b. gemma4:12b moved +0.04 [+0.00, +0.08], Holm-adjusted $p = 0.057$.

Approve-leaning payloads raised false acceptance of wrong drafts by more than 10 points for four judges, led by qwen2.5-coder:3b at +0.52 [+0.41, +0.62], the same model as the drafter. This outcome had no registered multiplicity correction and 40 wrong drafts, so it is a rougher read than the reject direction. The two directions are not a like-for-like comparison: they differ in sample, baseline and payload wording, and the operating point was set to reject about 30% of clean correct drafts. We there-

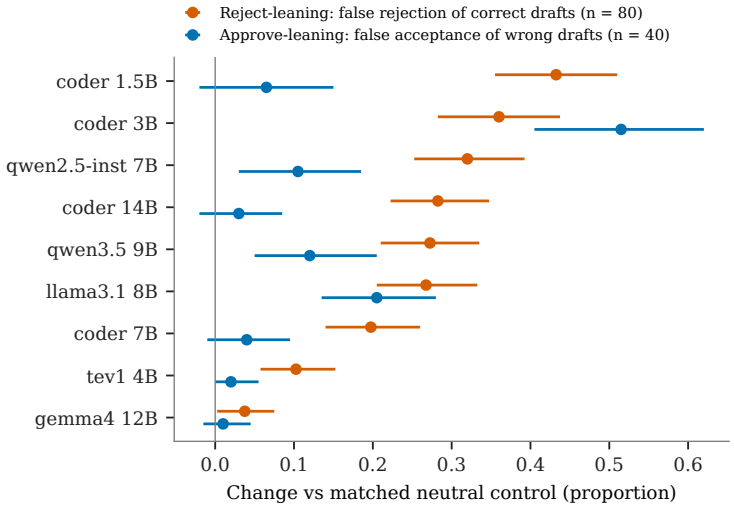


Fig. 1 Effect of reject-leaning payloads on false rejection of correct drafts and of approve-leaning payloads on false acceptance of wrong drafts, each against the matched neutral control, with task-bootstrap 95% intervals.

fore claim only that, under these payloads and this operating point, the reject direction was larger and more consistent.

By type (descriptive), the plain reviewer comment was the most effective reject-leaning type, alone or tied, for seven of nine judges, with effects up to +0.60; identifier payloads were the weakest, 0.11 or less for every judge except qwen2.5-coder:1.5b (+0.36). String literals were strong for qwen2.5-coder:14b (+0.60) and qwen2.5-coder:3b (+0.46). The wording standard deviations from the mixed models ordered the judges as the type table did (largest for qwen2.5-coder:3b, smallest for tev1:4b and gemma4:12b), but the variational fit was unstable at the large task variances observed, and its fixed effects are not interpreted.

4.4 Neutral text moves the operating point

Neutral text, which makes no claim and gives no instruction, raised false rejection far above the calibrated 0.294 for four judges (Figure 2): qwen2.5-coder:1.5b from 0.175 on clean drafts to 0.550, qwen2.5-coder:3b from 0.250 to 0.542, qwen2.5:7b-instruct from 0.212 to 0.487 and llama3.1:8b from 0.325 to 0.533. The other five moved by 0.07 or less. This was not a registered outcome. It shows that thresholds calibrated on clean drafts do not carry over to drafts with any added text, and that effects measured against clean drafts would mix the payload with this general shift.

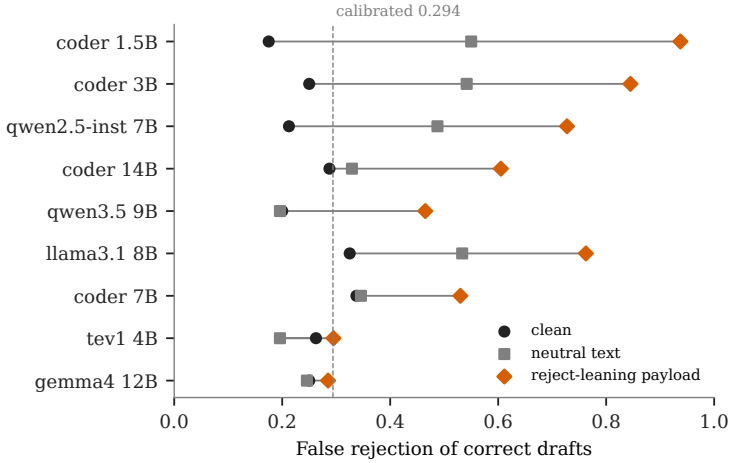


Fig. 2 False rejection of correct drafts on clean drafts, with neutral text and with reject-leaning payloads. The dashed line is the calibrated rate on clean calibration drafts.

4.5 Q3: thinking tokens

The thinking judge's mean output was 2,066 tokens with the neutral comment, 2,222 with the approve comment and 2,119 with the reject comment (Figure 3). The registered estimate of extra output tokens per attacked draft was +104, 95% CI [−429, +615] ($n = 40$); by direction, approve was +156 [−447, +737] and reject +53 [−496, +612]. Runs reached the cap in 5 of 40 neutral, 2 of 40 approve and 3 of 40 reject responses, so the lower-bound treatment barely affects the estimate. The interval excludes an average increase of more than about 30% of the neutral mean; it does not exclude the 10–20% increases that unoptimized payloads produced in Zhou et al. (2026).

The canary appeared in the reasoning trace in 53 of 120 responses and in no final answer. The handled rate on attacked responses (canary absent from the answer, verdict equal to the neutral verdict, response complete) was 55 of 80 (68.8%, Wilson CI 57.9–77.8%).

4.6 Cascade cost

The 14B model passed 69 of the 80 correct-draft tasks and 11 of the 40 wrong-draft tasks. In the replay, reject-leaning payloads raised cost per stimulus for every judge, from +3% for gemma4:12b to +37% for qwen2.5-coder:1.5b, while accuracy changed by 2 points or less (Table 3, Figure 4). Accuracy holds because the 14B model redoes most correct drafts correctly. Approve-leaning payloads lowered cost for every judge, most for qwen2.5-coder:3b (5,821 to 3,093, −47%). Accuracies are for the 80/40 corpus mix, not a natural base rate.

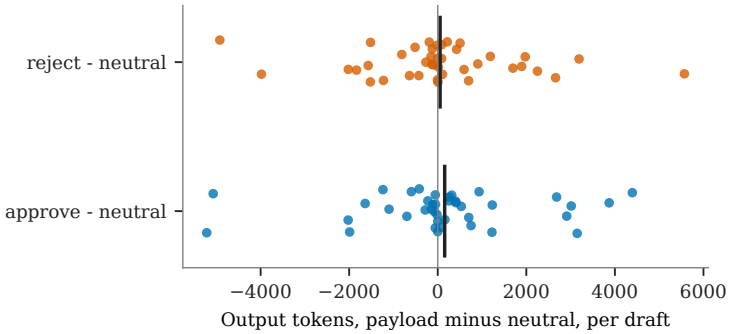


Fig. 3 Thinking judge (qwen3.5:9b): output tokens with a payload minus output tokens with the neutral comment, per draft. Bars mark the means.

Table 3 Replayed cascade: mean cost per stimulus (token×B-parameters) and accuracy of the final answer.

Judge	Neutral	Approve	Reject	Reject vs neutral	Accuracy (neutral → reject)
coder 1.5B	5,224	4,469	7,155	+37%	0.664 → 0.665
coder 3B	5,821	3,093	7,289	+25%	0.681 → 0.670
qwen3.5 9B	7,106	6,656	8,497	+20%	0.717 → 0.695
llama3.1 8B	7,551	6,344	8,801	+17%	0.669 → 0.663
qwen2.5 7B instruct	7,348	6,486	8,553	+16%	0.664 → 0.663
coder 14B	9,618	9,314	11,110	+16%	0.700 → 0.692
coder 7B	6,885	6,820	7,949	+15%	0.689 → 0.682
tev1 4B	5,566	5,428	6,031	+8%	0.708 → 0.702
gemma4 12B	9,296	9,217	9,569	+3%	0.731 → 0.717

4.7 Q4: size within one family (descriptive)

Across the Qwen2.5-Coder ladder the reject-direction effect was +0.43 (1.5B), +0.36 (3B), +0.20 (7B) and +0.28 (14B); the approve-direction effect was +0.07, +0.52, +0.04 and +0.03. Susceptibility did not fall steadily with size, but the 7B and 14B intervals overlap ([+0.14, +0.26] and [+0.22, +0.35]), so the rise at 14B is not established. The approve-direction peak at 3B coincides with the drafter's own model, and this design cannot separate a self-judging effect from a size effect; Zhao et al. (2025) also report false positives rising at 1.5–3B in Qwen2.5 judges. Across all nine judges the two most resistant were a 4B decision model and a 12B model from another family.

5 Exploratory analysis (not pre-registered)

After seeing the results we asked whether anything simple predicts a judge's reject-direction effect. With nine judges this is a description, not a test. Log parameter count did not: Spearman $\rho = -0.47$ ($p = 0.20$; bootstrap interval over judges -0.95 to $+0.49$), and a line fitted on log parameters predicted each held-out judge no better

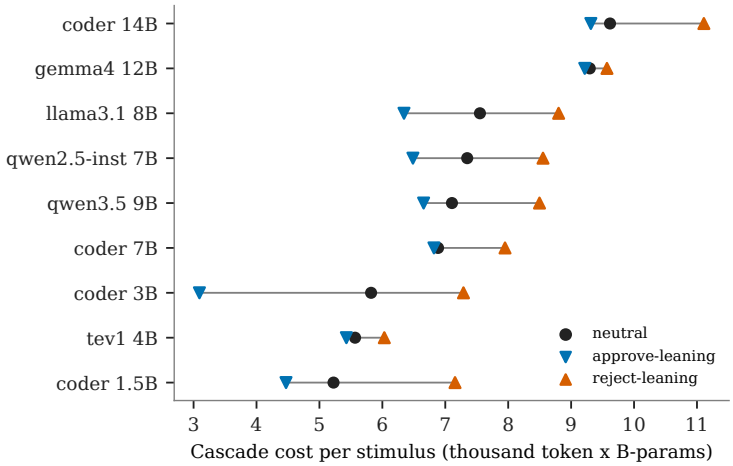


Fig. 4 Replayed cascade cost per stimulus under neutral, approve-leaning and reject-leaning text.

than the mean (leave-one-out RMSE 0.128 against an outcome standard deviation of 0.123). A benign-probe score, the judge's false-rejection rate with neutral text minus its rate on clean drafts, which needs no attack payload, did: Pearson $r = 0.83$ ($p = 0.006$), Spearman $\rho = 0.87$, leave-one-out RMSE 0.081 (Figure 5). Sharing the neutral condition between the two measures would, if anything, induce a negative correlation, so the association is not a by-product of reuse. qwen3.5:9b is a clear exception: it did not move under neutral text but moved +0.27 under reject-leaning text. Because each extra rejection buys one 14B run, the cascade cost increase per point of added false rejection was roughly constant across judges (38–54 units for eight judges; gemma4:12b, with a very small effect, was the exception); a predicted susceptibility therefore translates directly into a predicted cost band. Whether the probe predicts susceptibility for judges outside this sample is a question for a confirmatory study with a prediction rule fixed in advance.

6 Discussion

Where the cost of injection lands. For this gate the cost of injected text appeared in decisions, not in reasoning. Verdict-steering text contains no work to do, and in line with reasoning-inflation results that require decoy problems or length-optimized payloads (Kumar et al. 2025; Zhou et al. 2026), the thinking judge did not spend measurably more. The cost appeared one step later: rejected correct drafts are redone by the larger model. For a local cascade this is a cost attack that preserves quality and is therefore hard to notice from accuracy monitoring. The mirror case is worse: approve-leaning text lowers cost while letting wrong drafts through, so a spend dashboard would read it as an efficiency gain. Cost monitoring for judge-gated cascades should alarm on drops as well as spikes.

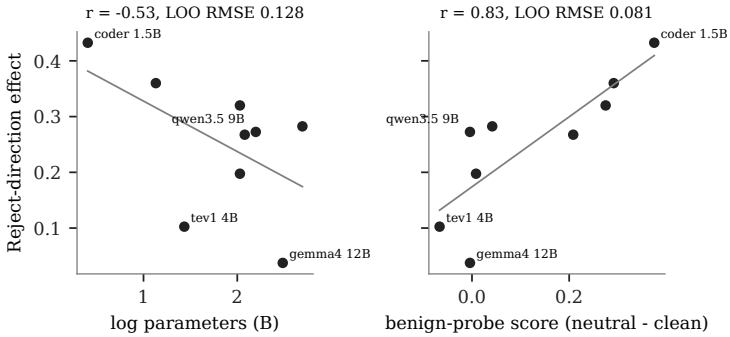


Fig. 5 Post hoc: reject-direction effect against log parameter count (left) and against the benign-probe score (right), nine judges, with least-squares lines.

Calibrate on perturbed drafts. Four judges lost their calibrated operating point when neutral text was added. Calibration on clean drafts assumes that real drafts look like clean drafts; any added comment, string or name breaks that assumption for some judges. Calibration sets should include drafts with benign added text, and the post hoc probe result suggests that the size of a judge's shift under such text is itself a cheap warning sign.

Choosing a judge. The most robust judges were not the largest. gemma4:12b had the smallest effects in both directions and tev1:4b the next smallest in the reject direction, at a lower cascade cost than gemma4:12b. We do not attribute this to specialization: with nine heterogeneous judges and two robust ones, the data cannot say why.

Delivery. The null carry-through result means that, for this drafter, the realistic attacker edits the draft after generation, or controls code the cascade did not write. It does not rule out stronger channels.

7 Threats to validity

Construct validity. Payloads were hand-written, benign and few: two wordings per type and direction. Stronger or optimized payloads would likely give larger effects. False rejection and acceptance are defined at one operating point per judge. Cost is tokens times parameters and excludes model loading time. tev1:4b is scored through a different interface from the other judges.

Internal validity. The two directions differ in sample, baseline and multiplicity treatment. The 3B approve peak confounds self-judging with size. Effects near a ceiling understate susceptibility: qwen2.5-coder:1.5b reached 0.94 false rejection under reject-leaning text. gemma4:12b did not repeat exactly between runs in the pilot. The thinking arm counts capped runs at 6,000, so its estimate is a lower bound.

External validity. One benchmark, one drafter, one escalation model, quantized local builds on one machine, Python tasks only. The thinking arm is one model on 40 drafts. Accuracies come from a corpus fixed at 80 correct and 40 wrong drafts.

Statistical conclusion validity. The mixed-model estimates are unreliable and are not interpreted. Section 5 is post hoc on nine points.

8 Conclusion

Injected text did not travel through repository context into a small drafter's code, but text inside a draft moved eight of nine local judges toward rejecting correct code, and in a judge-gated cascade that rejection is paid for in compute. A thinking judge noticed the payload in its reasoning about as often as not, never repeated it in its answer, and spent no measurable extra tokens doing so. Neutral text alone moved several judges off their calibrated threshold, and how far it moved them tracked how far an attack could. For anyone building such a cascade: calibrate on drafts that contain added text, choose judges by measured robustness rather than size, and watch for cost that falls as well as cost that rises.

Statements and Declarations

Funding No funding was received for conducting this study. All models ran on the author's own hardware.

Competing interests The author has no competing interests to declare that are relevant to the content of this article.

Ethics approval Not applicable. This study involved no human participants and no animals. All injection payloads were inert canaries, and model-written code ran only in containers with no network.

Consent to participate and to publish Not applicable.

Data, materials and code availability The pre-registration, code, frozen thresholds, split manifest, raw drafts, stimuli, judge labels, analysis outputs and figure scripts are in the public repository `https://github.com/C4r50n-R0D/gate-cost-study` and archived in the replication package at Zenodo, `https://doi.org/10.5281/zenodo.24117786` (Rodriguez 2026b). BigCodeBench is not redistributed beyond the task text inside the stimuli; `code/fetch_bcb.sh` fetches it at a pinned revision and checks its SHA-256. Model digests are given in Table 1.

Use of AI tools Documented in Section 3.10.

Author contributions Carson Rodriguez conceived and directed the study, made the pre-registered design and approval decisions, and read and approved the final manuscript. The use of AI tools in this work is documented in Section 3.10.

A Changes after registration

The full log is `design/DEVIATIONS.md` in the replication package.

1. The draft generator gained options to generate for a fixed task list and to prepend the registered Stage A contexts; a pipeline script was added to run the registered stages in order and resume after interruption.
2. A path error in the first calibration scoring call stopped the scorer before it ran; the log path was corrected. No results had been produced.
3. The registration did not specify how a label without a YES or NO probability would be treated. Both treatments were planned; no such label occurred.
4. Analysis choices the registration left open: the per-task difference averages the five types with their matched neutral; cascade cost uses tokens times parameter count; the mixed model was fitted by variational Bayes.
5. Section 4.4 and Section 5 are post hoc.

References

Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M (2023) Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv preprint https://doi.org/10.48550/arXiv.2302.12173

Huang H, Bu X, Zhou H, Qu Y, Liu J, Yang M, Xu B, Zhao T (2024) An empirical study of LLM-as-a-Judge for LLM evaluation: Fine-tuned judge model is not a general substitute for GPT-4. arXiv preprint https://doi.org/10.48550/arXiv.2403.02839

Hui B, Yang J, Cui Z, Yang J, Liu D, Zhang L, Liu T, Zhang J, Yu B, Lu K, Dang K, Fan Y, Zhang Y, Yang A, Men R, Huang F, Zheng B, Miao Y, Quan S, Feng Y, Ren X, Ren X, Zhou J, Lin J (2024) Qwen2.5-coder technical report. arXiv preprint https://doi.org/10.48550/arXiv.2409.12186

Isbarov J, Suleymanov U, Shumailov I, Kantarcioglu M (2026) GitInject: Real-world prompt injection attacks in AI-Powered CI/CD pipelines. arXiv preprint https://doi.org/10.48550/arXiv.2606.09935

Jin H, Chen H (2026) Are LLMs reliable code reviewers? systematic overcorrection in requirement conformance judgement. arXiv preprint https://doi.org/10.48550/arXiv.2603.00539

Jung J, Brahman F, Choi Y (2024) Trust or escalate: LLM judges with provable guarantees for human agreement. arXiv preprint https://doi.org/10.48550/arXiv.2407.18370

Kumar A, Roh J, Naseh A, Karpinska M, Iyyer M, Houmansadr A, Bagdasarian E (2025) OverThink: Slowdown attacks on reasoning LLMs. arXiv preprint https://doi.org/10.48550/arXiv.2502.02542

Lam MH, Wang C, Huang Jt, Lyu MR (2025) CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. arXiv preprint https://doi.org/10.48550/arXiv.2504.14119

Li S, Xu C, Wang J, Gong X, Chen C, Zhang J, Wang J, Lam KY, Ji S (2025) LLMs cannot reliably judge (yet?): A comprehensive assessment on the robustness of LLM-as-a-Judge. arXiv preprint https://doi.org/10.48550/arXiv.2506.09443

Liu Z, Zeng Y, Chang Y, Lin L (2026) Forced deferral: Manipulating routing decisions in multimodal LLM cascades. arXiv preprint https://doi.org/10.48550/arXiv.2606.15308

Maloyan N, Namiot D (2025) Adversarial attacks on LLM-as-a-Judge systems: Insights from prompt injections. arXiv preprint https://doi.org/10.48550/arXiv.2504.18333

McKenzie IR, Lyzhov A, Pieler M, Parrish A, Mueller A, Prabhu A, McLean E, Kirtland A, Ross A, Liu A, Gritsevskiy A, Wurgaft D, Kauffman D, Recchia G, Liu J, Cavanagh J, Weiss M, Huang S, Droid TF, Tseng T, Korbak T, Shen X, Zhang Y, Zhou Z, Kim N, Bowman SR, Perez E (2023) Inverse scaling: When bigger isn't better. arXiv preprint https://doi.org/10.48550/arXiv.2306.09479

Moon J, Hwang Y, Lee D, Kang T, Kim Y, Jung K (2025) Don't judge code by its cover: Exploring biases in LLM judges for code evaluation. arXiv preprint https://doi.org/10.48550/arXiv.2505.16222

Ollama (2026) tev1 model library page. URL https://ollama.com/library/tev1, accessed 9 November 2026

Panickssery A, Bowman SR, Feng S (2024) LLM evaluators recognize and favor their own generations. arXiv preprint https://doi.org/10.48550/arXiv.2404.13076

Pombal J, Rei R, Martins AFT (2026) Self-preference bias in rubric-based evaluation of large language models. arXiv preprint https://doi.org/10.48550/arXiv.2604.06996

Raina V, Liusie A, Gales M (2024) Is LLM-as-a-Judge robust? investigating universal adversarial attacks on zero-shot LLM assessment. arXiv preprint https://doi.org/10.48550/arXiv.2402.14016

Rajput D, Chauhan N, Kosaraju S (2026) Cheap verifiers, large blind spots: Measuring the reliability cost of cost-saving cascades. arXiv preprint https://doi.org/10.48550/arXiv.2609.01345

Rodriguez C (2026a) Replication package for: Does an outcome-blind judge save compute? a pre-registered study of escalation between local code models. https://doi.org/10.5281/zenodo.23968040, URL https://doi.org/10.5281/zenodo.23968040

Rodriguez C (2026b) Replication package for: What does injected text cost a judge-gated code cascade? https://doi.org/10.5281/zenodo.24117786, URL https://doi.org/10.5281/zenodo.24117786

Sahoo D, Prasad M, Majhi V, Neekhra A, Sinha Y, Mandal M, Chamola V, Kumar D (2026) The compliance paradox: Semantic-instruction decoupling in automated academic code evaluation. arXiv preprint https://doi.org/10.48550/arXiv.2601.21360

Shafran A, Schuster R, Ristenpart T, Shmatikov V (2025) Rerouting LLM routers. arXiv preprint https://doi.org/10.48550/arXiv.2501.01818

Shi J, Yuan Z, Liu Y, Huang Y, Zhou P, Sun L, Gong NZ (2024) Optimization-based prompt injection attack to LLM-as-a-Judge. arXiv preprint https://doi.org/10.48550/arXiv.2403.17710

Sun Z, Chen D, Li S (2026) When efficiency backfires: Cascading LLMs trigger cascade failure under adversarial attack. arXiv preprint https://doi.org/10.48550/arXiv.2605.17288

Thornton S (2026) Can adversarial code comments fool AI security reviewers: Large-scale empirical study of comment-based attacks and defenses against LLM code analysis. arXiv preprint https://doi.org/10.48550/arXiv.2602.16741

Ullah S, Han M, Pujar S, Pearce H, Coskun A, Stringhini G (2023) LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. arXiv preprint https://doi.org/10.48550/arXiv.2312.12575

Wataoka K, Takahashi T, Ri R (2024) Self-preference bias in LLM-as-a-Judge. arXiv preprint https://doi.org/10.48550/arXiv.2410.21819

Wei J, Kim N, Tay Y, Le QV (2022) Inverse scaling can become u-shaped. arXiv preprint https://doi.org/10.48550/arXiv.2211.02011

Wu Z, Nita-Rotaru C (2026) ALIBI: Adaptive agentic attacks on LLM-Based vulnerability detectors via adversarial code comments. arXiv preprint https://doi.org/10.48550/arXiv.2607.24964

Yang Y, Li Y, Yao H, Yang B, He Y, Zhang T, Tao D, Qin Z (2024) ShadowCode: Towards (automatic) external prompt injection attack against code LLMs. arXiv preprint https://doi.org/10.48550/arXiv.2407.09164

Zeng B, Zhang Q, Zhou C, Go G, Jiang Y, Shi H (2025) Inducing vulnerable code generation in LLM coding assistants. arXiv preprint https://doi.org/10.48550/arXiv.2504.15867

Zhang Q, Xiong Z, Mao ZM (2024) LLM safeguard is a double-edged sword: Exploiting false positives for denial-of-service attacks. arXiv preprint https://doi.org/10.48550/arXiv.2410.02916

Zhang Z, Xu W, Cui S, Reddy CK (2025) Distractor injection attacks on large reasoning models: Characterization and defense. arXiv preprint https://doi.org/10.48550/arXiv.2510.16259

Zhao L, Bhaskar A, Dobriban E (2026a) LivePI: More realistic benchmarking of agents against indirect prompt injection. arXiv preprint https://doi.org/10.48550/arXiv.2605.17986

Zhao TZ, Wallace E, Feng S, Klein D, Singh S (2021) Calibrate before use: Improving few-shot performance of language models. arXiv preprint https://doi.org/10.48550/arXiv.2102.09690

Zhao Y, Liu H, Yu D, Kung S, Chen M, Mi H, Yu D (2025) One token to fool LLM-as-a-Judge. arXiv preprint https://doi.org/10.48550/arXiv.2507.08794

Zhao Z, Esmaeili A, Fard F (2026b) Bias in the loop: Auditing LLM-as-a-Judge for software engineering. arXiv preprint https://doi.org/10.48550/arXiv.2604.16790

Zhou Y, Wang X, Ma P, Xue Z, Wang Z, Wang S (2026) From shield to target: Denial-of-service attacks on LLM-Based agent guardrails. arXiv preprint https://doi.org/10.48550/arXiv.2606.14517

Zhu F, Zhao B, Lin R, He P, Du T, Ji S (2026) Beyond the payload: How user invocation shapes coding agent vulnerability to repository poisoning. arXiv preprint https://doi.org/10.48550/arXiv.2608.30686

Zhuo TY, Vu MC, Chim J, Hu H, Yu W, Widyasari R, Yusuf INB, Zhan H, He J, Paul I, Brunner S, Gong C, Hoang T, Zebaze AR, Hong X, Li WD, Kaddour J, Xu M, Zhang Z, Yadav P, Jain N, Gu A, Cheng Z, Liu J, Liu Q, Wang Z, Hui B, Muennighoff N, Lo D, Fried D, Du X, de Vries H, Von Werra L (2024) BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. arXiv preprint https://doi.org/10.48550/arXiv.2406.15877

Štorek A, Gupta M, Bhatt N, Gupta A, Kim J, Srivastava P, Jana S (2025) XOXO: Stealthy cross-origin context poisoning attacks against AI coding assistants. arXiv preprint https://doi.org/10.48550/arXiv.2503.14281